STAT 413, Adam Optimization

and other thoughts on Stochastic Approximation

9 March 2026

Recap on Stochastic Approximation

Last week, we considered two methods from the 1950's that search for a global minimum of a convex (noisy) loss function $Q(\theta, v)$.

Robbins-Monroe

- Update Rule:

$$\theta_{k+1} \leftarrow \theta_k - a_k y(\theta_k)$$

- $y(\theta)$ is the gradient of $Q(\theta, v)$ with respect to θ.
- v is noise added into the loss / gradient.

Recap on Stochastic Approximation

Last week, we considered two methods from the 1950's that search for a global minimum of a convex (noisy) loss function $Q(\theta, v)$.

Robbins-Monroe

- Update Rule:

$$\theta_{k+1} \leftarrow \theta_k - a_k y(\theta_k)$$

- $y(\theta)$ is the gradient of $Q(\theta, v)$ with respect to θ.
- v is noise added into the loss / gradient.

Kiefer-Wolfowitz

- Update Rule:

$$\theta_{k+1} \leftarrow \theta_k - a_k \hat{y}(\theta_k)$$

- $\hat{y}(\theta)$ is an estimate of the gradient of $Q(\theta, v)$ based on $2d$ evaluations of the loss function.

The Adam Optimizer

Now, we jump to 2014–2017, with *Adaptive moment estimation* or Adam. Currently, it is referred to as

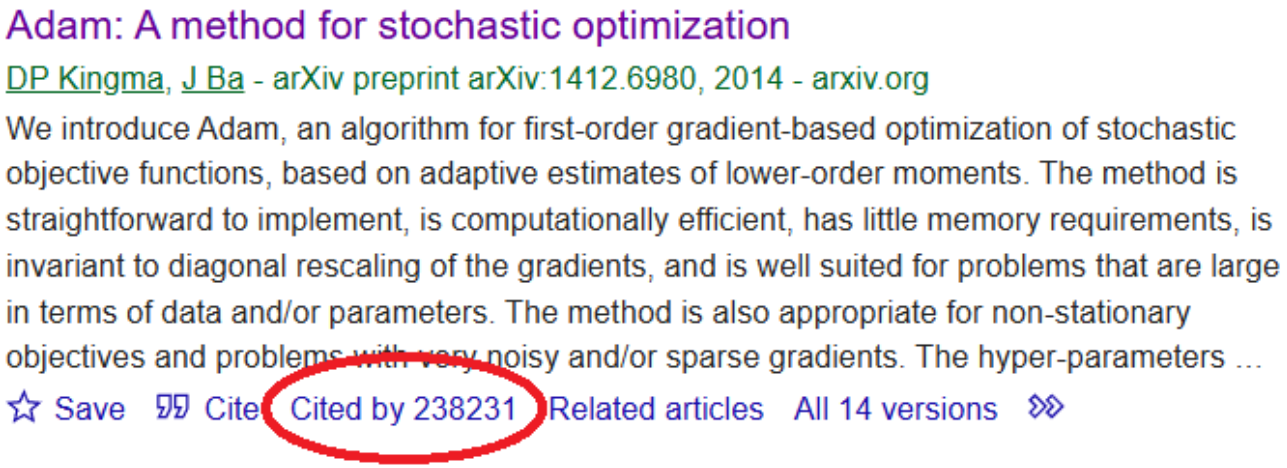
> *a cornerstone optimization algorithm in deep learning, widely recognized for its flexibility with adaptive learning rates and efficiency in handling large-scale data*

The Adam Optimizer

Now, we jump to 2014–2017, with *Adaptive moment estimation* or Adam. Currently, it is referred to as

> *a cornerstone optimization algorithm in deep learning, widely recognized for its flexibility with adaptive learning rates and efficiency in handling large-scale data*

Ultimately, Adam is a gradient-based optimizer similar in spirit to Robbins-Monroe, but very popular in ML and AI.


Adam: A method for stochastic optimization

DP Kingma, J Ba - arXiv preprint arXiv:1412.6980, 2014 - arxiv.org

We introduce Adam, an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. The method is straightforward to implement, is computationally efficient, has little memory requirements, is invariant to diagonal rescaling of the gradients, and is well suited for problems that are large in terms of data and/or parameters. The method is also appropriate for non-stationary objectives and problems with very noisy and/or sparse gradients. The hyper-parameters ...

Save Cite Cited by 238231 Related articles All 14 versions

The Adam Optimizer

Terms and Definitions

- α, a step-size or "learning rate"
- ε, a small "stabilizing" term
- $\beta_1, \beta_2 \in [0, 1)$, exponential decay rates
- $y(\theta)$, the gradient
- m_k, a 1st moment
- v_k, a 2nd moment

The Adam Optimizer

Terms and Definitions

- α, a step-size or "learning rate"
- ε, a small "stabilizing" term
- $\beta_1, \beta_2 \in [0, 1)$, exponential decay rates
- $y(\theta)$, the gradient
- m_k, a 1st moment
- v_k, a 2nd moment

Algorithm

- Begin with some θ_0, and $m_0 = v_0 = 0$.
- Update

$$
\begin{aligned}
m_k &\leftarrow \beta_1 m_{k-1} + (1 - \beta_1) y(\theta_k) \\
v_k &\leftarrow \beta_2 v_{k-1} + (1 - \beta_2) y(\theta_k)^2 \\
\hat{m}_k &\leftarrow m_k / (1 - \beta_1^k) \\
\hat{v}_k &\leftarrow v_k / (1 - \beta_2^k) \\
\theta_k &\leftarrow \theta_{k-1} - \alpha \hat{m}_k / (\sqrt{\hat{v}_k} + \varepsilon)
\end{aligned}
$$

Adam vs Robbins-Monroe

Robbins-Monroe

- Choose a gain sequence (step size) $a_k \to 0$ such that $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$.
- Just compute and use the current gradient $y(\theta_k)$ at each θ_k.

Adam

- Choose a learning rate (step size) $\alpha > 0$.
- Maintain a moving average (see STAT 479) of computed gradients $m_{k+1} \leftarrow \beta_1 m_k + (1 - \beta_1) y(\theta_k)$.
- β_1 tells Adam how quickly to "forget" past gradients in favour of the new gradient.

Adam vs Robbins-Monroe

Robbins-Monroe

- Choose a gain sequence (step size) $a_k \to 0$ such that $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$.
- Just compute and use the current gradient $y(\theta_k)$ at each θ_k.

Adam

- Choose a learning rate (step size) $\alpha > 0$.
- Maintain a moving average (see STAT 479) of computed gradients $m_{k+1} \leftarrow \beta_1 m_k + (1 - \beta_1) y(\theta_k)$.
- β_1 tells Adam how quickly to "forget" past gradients in favour of the new gradient.

What happens if we set $\beta_1 = \beta_2 = 0$?

Adam with $\beta_1 = \beta_2 = 0$ (bad idea!)

Living in the current moment

Without any influence on past gradient calculations, the update rule becomes

$$\theta_{k+1} \leftarrow \theta_k - \frac{\alpha}{|y(\theta_k)| + \varepsilon} y(\theta_k),$$

which is like Robbins-Monroe with gain sequence

$$a_k = \frac{\alpha}{|y(\theta_k)| + \varepsilon}.$$

Note however that we don't want to do this as a_k will not go to zero as required by RM.

Adam with $\beta_1 = \beta_2 = 0$ (bad idea!)

Living in the current moment

Without any influence on past gradient calculations, the update rule becomes

$$\theta_{k+1} \leftarrow \theta_k - \frac{\alpha}{|y(\theta_k)| + \varepsilon} y(\theta_k),$$

which is like Robbins-Monroe with gain sequence

$$a_k = \frac{\alpha}{|y(\theta_k)| + \varepsilon}.$$

Note however that we don't want to do this as a_k will not go to zero as required by RM.

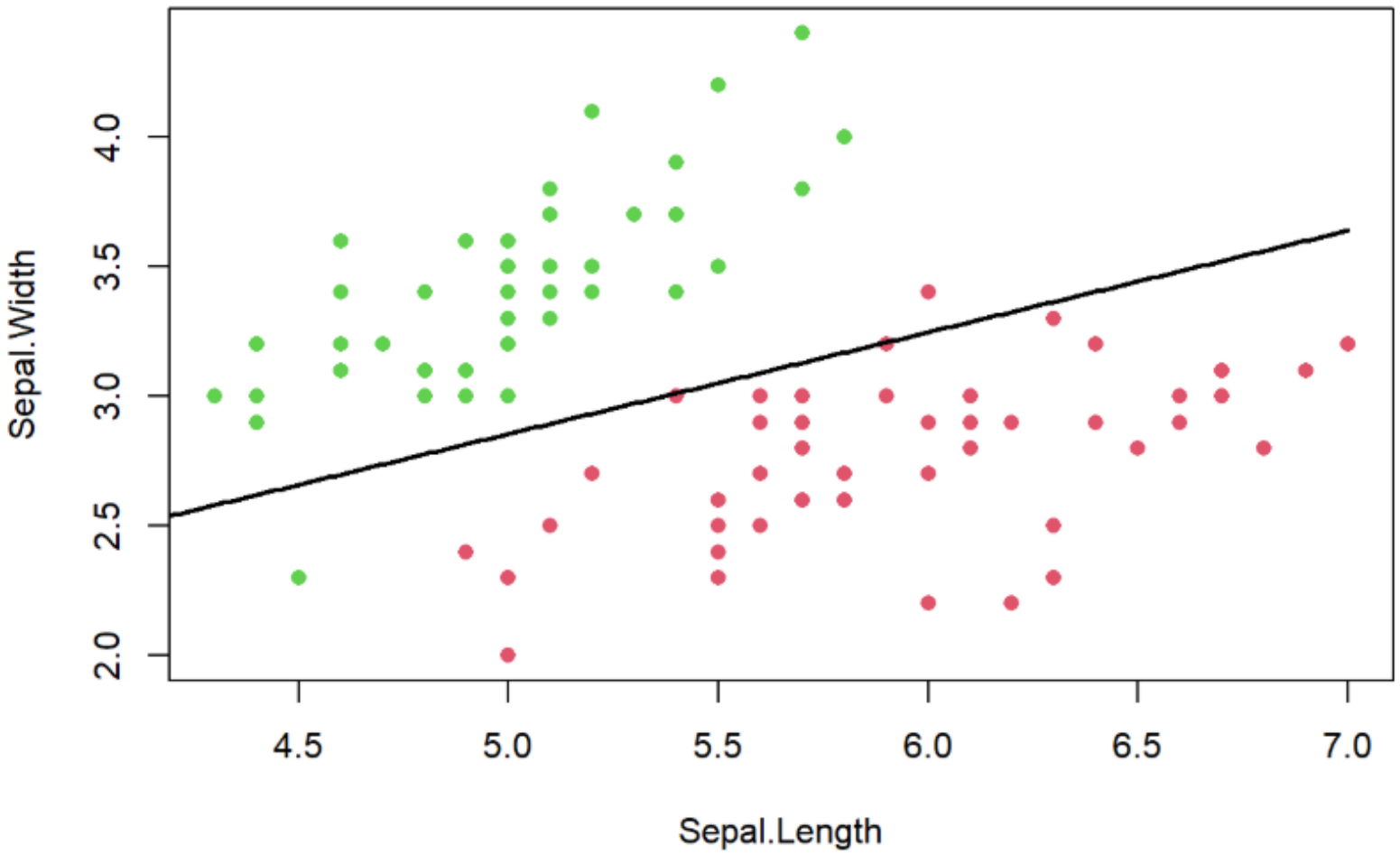
Fun Fact!

Indeed, usually β_1 and β_2 are close to 1 such as 0.9 and 0.999, respectively.

The Soft-Margin Support Vector Machine

basically draw a line to separate the green points from the red points

Fisher's Iris Data



The Soft-Margin Support Vector Machine

basically draw a line to separate the green points from the red points

The inputs are $x_j \in \mathbb{R}^d$ and the output is $y_j \in \{\pm 1\}$. The Loss function is

$$L(\theta) = \frac{1}{n} \sum_{j=1}^{n} \max\{0, 1 - y_j(\theta \cdot x_j - \theta_0)\} + \lambda \sum_{i=1}^{d} \theta_i^2$$

where λ is a tuning parameter. The derivative for θ_i with $i = 1, \ldots, n$ is

$$2\lambda\theta_i + \frac{1}{n} \sum_{j=1}^{n} \begin{cases} 0 \\ -y_j x_{j,i} \end{cases}$$

and for θ_0

$$\frac{1}{n} \sum_{j=1}^{n} \begin{cases} 0 \\ y_j \end{cases}$$

The Soft-Margin Support Vector Machine

basically draw a line to separate the green points from the red points

The inputs are $x_j \in \mathbb{R}^d$ and the output is $y_j \in \{\pm 1\}$. The Loss function is

$$L(\theta) = \frac{1}{n} \sum_{j=1}^{n} \max\{0, 1 - y_j(\theta \cdot x_j - \theta_0)\} + \lambda \sum_{i=1}^{d} \theta_i^2$$

where λ is a tuning parameter.

The first term

$$\frac{1}{n} \sum_{j=1}^{n} \max\{0, 1 - y_j(\theta \cdot x_j - \theta_0)\}$$

is a loss function for the number of points on the "wrong" side of the line / separating hyperplane.

The Soft-Margin Support Vector Machine

basically draw a line to separate the green points from the red points

The inputs are $x_j \in \mathbb{R}^d$ and the output is $y_j \in \{\pm 1\}$. The Loss function is

$$L(\theta) = \frac{1}{n} \sum_{j=1}^{n} \max\{0, 1 - y_j(\theta \cdot x_j - \theta_0)\} + \lambda \sum_{i=1}^{d} \theta_i^2$$

where λ is a tuning parameter.

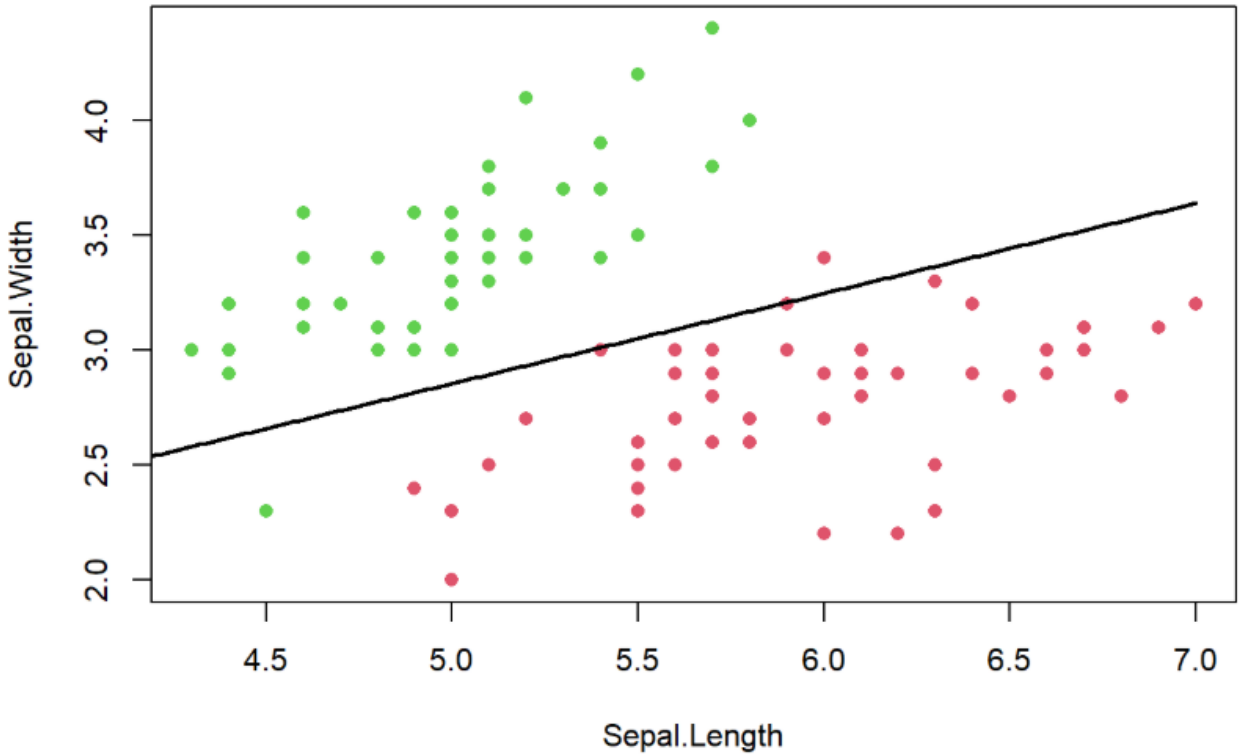
The second term

$$\lambda \sum_{i=1}^{d} \theta_i^2$$

is a penalty for using large θ_i values similar to Ridge regression.

SVM with Robbins-Monroe

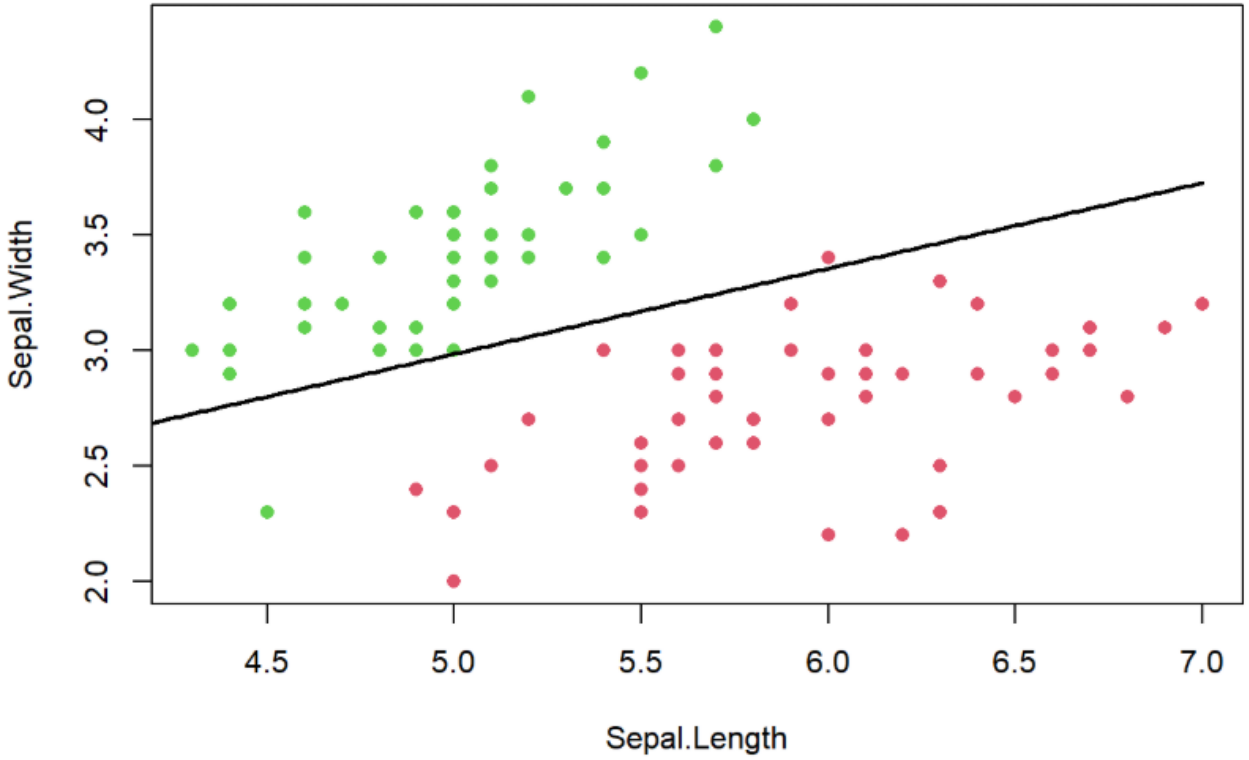
Fisher's Iris Data



Accuracy: 2 out of 100 wrong, and 2 out of 100 on the line

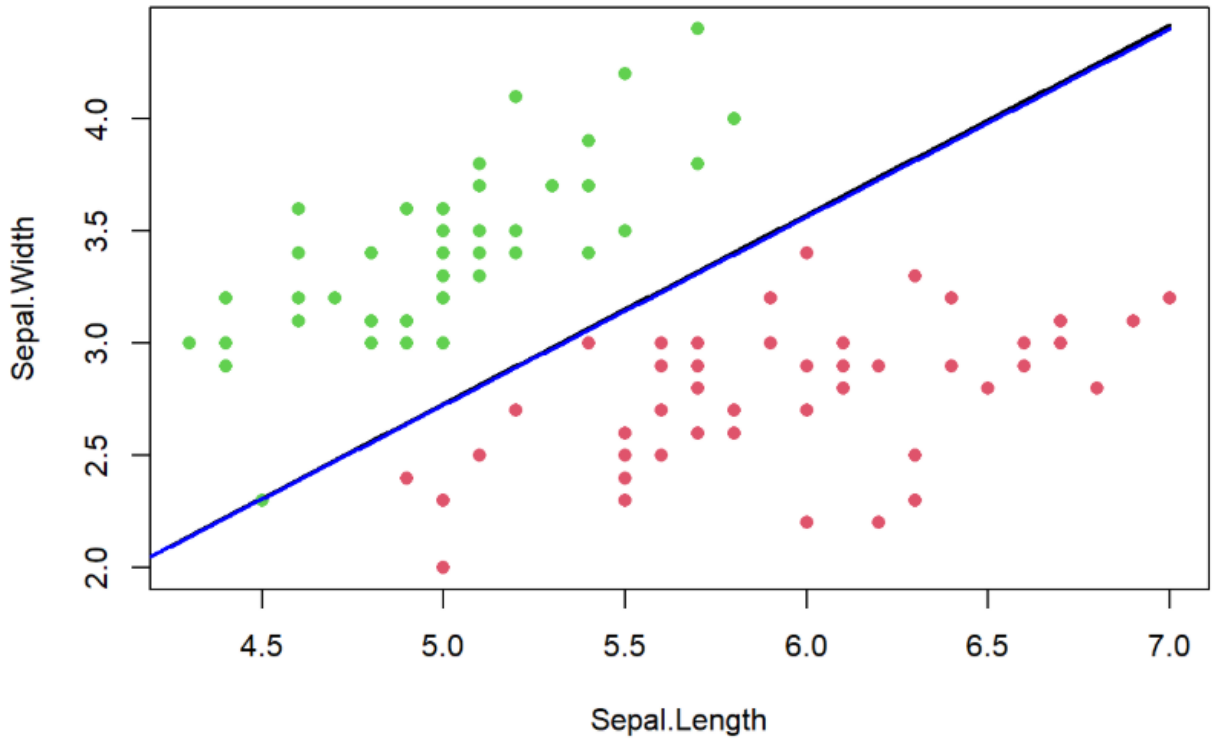
SVM with Kiefer-Wolfowitz

Fisher's Iris Data



Accuracy: 3 out of 100 wrong

SVM with Adam

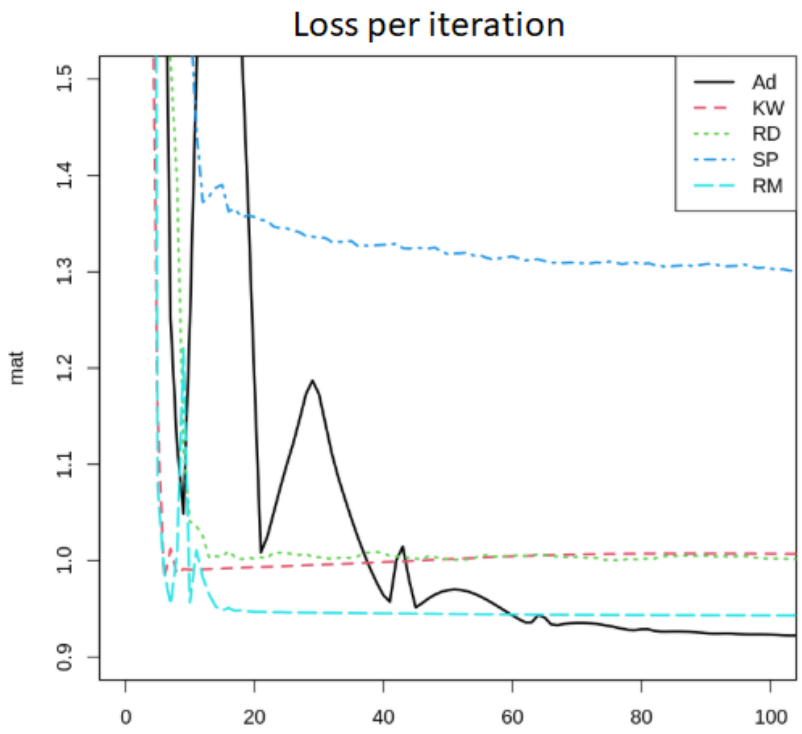


Accuracy: 0 out of 100 wrong, and 1 out of 100 on the line

Comparison of Methods

In this simulation

- Robbins-Monroe dropped the fastest after 10-20 iterations.
- Adam eventually (after 100 iterations) achieved to smallest loss.
- The approximate gradient methods like KW performed worse.



Summary of Stochastic Approximation

- There are many different methods beyond what we discussed in class.

Summary of Stochastic Approximation

- There are many different methods beyond what we discussed in class.
- Roughly, there are gradient-based and gradient-free methods, but also optimizers based on higher order derivatives.

Summary of Stochastic Approximation

- There are many different methods beyond what we discussed in class.
- Roughly, there are gradient-based and gradient-free methods, but also optimizers based on higher order derivatives.
- No one method is perfect for all your problems.

Summary of Stochastic Approximation

- There are many different methods beyond what we discussed in class.
- Roughly, there are gradient-based and gradient-free methods, but also optimizers based on higher order derivatives.
- No one method is perfect for all your problems.
- But Adam and related methods are very popular in model machine learning.

Summary of Stochastic Approximation

- There are many different methods beyond what we discussed in class.
- Roughly, there are gradient-based and gradient-free methods, but also optimizers based on higher order derivatives.
- No one method is perfect for all your problems.
- But Adam and related methods are very popular in model machine learning.
- See, for example, https://en.wikipedia.org/wiki/Stochastic_gradient_descent and the references therein.

Summary of Stochastic Approximation

Also, note that many of these classic statistical and machine learning methods are not fit with RM, KW, Adam, or similar methods. For example, in R,

- Support Vector Machines are usually fit via Quadratic Programming, https://en.wikipedia.org/wiki/Quadratic_programming
- Logistic regression and other GLMs are fit via iteratively reweighted least squares https://en.wikipedia.org/wiki/Iteratively_reweighted_least_squares
- As for myself (ABK), I often default to `optim()` in R as a first try at fitting some model to data.

But stochastic approximation is still very widely applicable to many model fitting problems.